

Docker Compose, Volume, Networks

01 August 2026 20:53

Here are beginner-friendly explanations of the remaining Docker topics.

6. Docker Compose

Definition

Docker Compose is a tool used to **run multiple Docker containers together** using a single YAML file (docker-compose.yml).

Instead of starting each container manually, Docker Compose starts all of them with one command.

Why do we need it?

Imagine a Spring Boot application.

It needs:

- Spring Boot Container
- MySQL Container
- Redis Container

Without Docker Compose:

docker run ...

docker run ...

docker run ...

You need to start each container separately.

With Docker Compose:

docker compose up

Everything starts automatically.

Real Example

A Spring Boot project consists of
Spring Boot

|



MySQL Database

|



Redis Cache

Docker Compose starts all three containers together.

Example docker-compose.yml

```
version: '3.8'
services:
  app:
    image: springboot-app
    ports:
      - "8080:8080"
  mysql:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
  redis:
    image: redis
Run
```

docker compose up
Everything starts.

Advantages

Without Compose	With Compose
Multiple commands	One command
Difficult	Easy
Manual networking	Automatic
Hard to manage	Simple

Common Commands

Start
docker compose up
Run in background
docker compose up -d
Stop
docker compose down

7. Docker Volumes

Definition

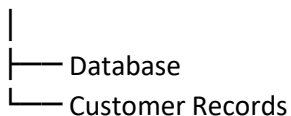
Docker Volumes store data permanently.
Normally,
Container deleted
↓
All data deleted.
Volumes prevent data loss.

Why?

Suppose MySQL stores customer data.

Without Volume

Container



Delete container

↓

Everything disappears.

With Volume

Container



Docker Volume



Customer Data

Delete container

↓

Data still exists.

Create another container



Same data comes back.

Example

Without Volume

Container



Users

Products

Orders

Delete container



Everything gone.

With Volume

Container



Volume



Users

Products

Orders

Delete container



Data remains.

Command

Create volume

```
docker volume create myvolume
```

Run

```
docker run -v myvolume:/var/lib/mysql mysql
```

Advantages

- Permanent storage
- Database backup
- Share data between containers
- Easy migration

8. Docker Networking

Definition

Docker Networking allows containers to communicate with each other.

Think of it like computers connected through a LAN.

Example

Spring Boot Container



Docker Network



MySQL Container
Spring Boot connects to MySQL using the network.

Without networking
Spring Boot
Cannot find
MySQL

With networking
Spring Boot
|
▼
Docker Network
|
▼
MySQL
Communication works.

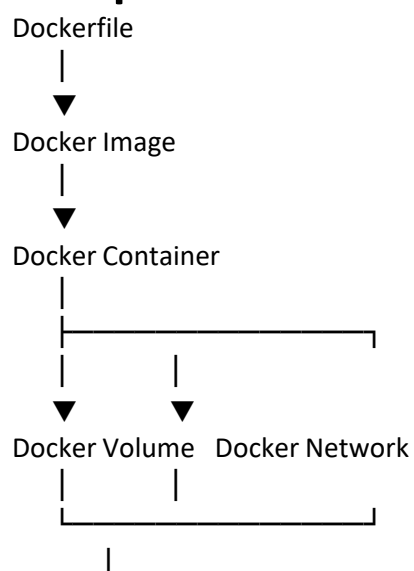
Example Command

Create network
docker network create mynetwork
Run MySQL
docker run --network mynetwork mysql
Run Spring Boot
docker run --network mynetwork springboot-app
Now both containers can communicate.

Types of Docker Networks

Type	Usage
Bridge	Default network for containers on one machine
Host	Container uses the host machine's network directly
None	No network access
Overlay	Connects containers across multiple Docker hosts (Docker Swarm)
Macvlan	Gives a container its own IP address on the physical network

Complete Docker Flow





Docker Compose
(Manages all containers together)

Interview Summary

Topic	One-line Definition	Example
Docker Compose	Runs multiple containers using one YAML file	Spring Boot + MySQL + Redis
Docker Volumes	Permanent storage for containers	Store MySQL database files
Docker Networking	Enables containers to communicate	Spring Boot connects to MySQL
Docker Image	Blueprint/template for containers	Spring Boot image
Docker Container	Running instance of an image	Running Spring Boot app
Dockerfile	File containing image build instructions	FROM openjdk:21 + COPY app.jar

For Spring Boot developers, a common real-world setup is:

- **Dockerfile** packages your Spring Boot application.
- **Docker Compose** starts the Spring Boot app, MySQL, and Redis together.
- **Docker Network** lets the application connect to MySQL and Redis.
- **Docker Volume** preserves your database data even if the MySQL container is removed.